APM52009 – Machine Learning for Scientific Computing and Numerical Analysis

Week 5: Physics-Informed Neural Networks

Victorita Dolean and Hadrien Montanelli

Abstract

Physics-Informed Neural Networks (PINNs) combine machine learning with physics-based models by embedding the laws of physics—typically expressed as PDEs—directly into the training of neural networks. This paradigm offers an attractive alternative to the classical numerical solvers presented in Chapter 4 (finite differences and finite elements), by turning the problem of solving a PDE into an optimization problem over neural-network parameters.

Contents

1 Vanilla PINNs **2**

1.1 Methodology 2

1.2 Theoretical results 5

2 Training dynamics of PINNs **7**

2.1 Linearized dynamics 7

2.2 Neural Tangent Kernel theory 8

2.3 Other practical considerations 9

3 The Deep Ritz method **10**

4 Automatic differentiation **12**

5 Exercises **14**

6 Suggested reading **15**

1 Vanilla PINNs

Historical note The idea of representing the solution of a differential equation by a neural network is not new: it was first explored by Lagaris, Likas, and Fotiadis in the late 1990s [6]. Their pioneering work introduced neural-network trial functions that automatically satisfy boundary conditions by design, called today the "hard enforcement" approach. The modern formulation of PINNs, proposed by Raissi, Perdikaris, and Karniadakis in 2019 [12], generalized this concept by combining the PDE residual, the boundary conditions, and possibly measurement data into a single loss function trained by SGD/Adam with backpropagation.

General idea More concretely, a neural network u_θ serves as a flexible trial function. Automatic differentiation gives exact derivatives such as $\partial_x u_\theta$ or $\partial_{xx} u_\theta$. The network is trained so that the PDE residual $r_\theta = \mathcal{N}(u_\theta) - f$ and the mismatch with boundary/initial conditions are jointly minimized. Today, PINNs form an important subfield of SciML, bridging data-driven modeling and numerical analysis.

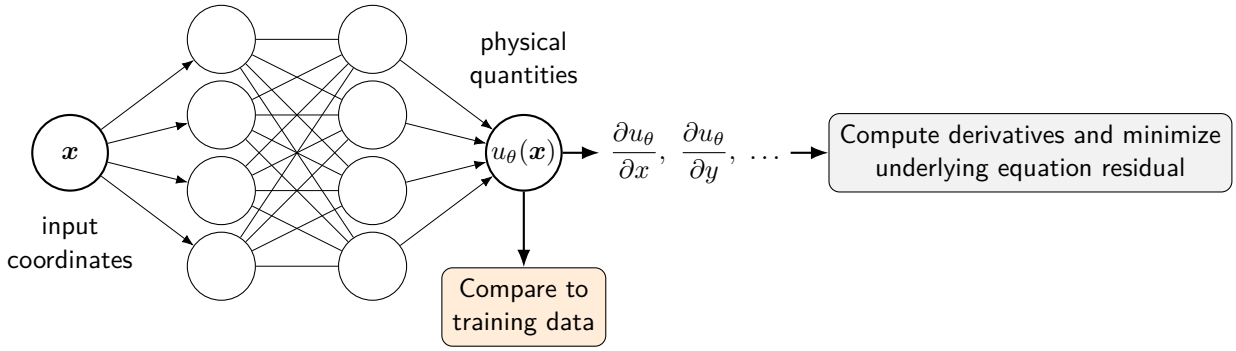


Figure 1: A minimal PINN schematic—the network outputs $u_\theta(\boldsymbol{x})$, automatic differentiation supplies the derivatives $\partial u_\theta/\partial x, \partial u_\theta/\partial y, \ldots$ to build residual losses, while observations (if any) are compared to u_θ.

We now describe the standard or *vanilla* formulation of PINNs. This method captures the essence of how neural networks can approximate the solution of PDEs by minimizing a *residual loss* while optionally penalizing the mismatch at boundaries or initial conditions.

1.1 Methodology

Let $D \subset \mathbb{R}^d$ be an open, bounded set with a smooth boundary. We consider a general boundary value problem

$$\begin{cases} \mathcal{N}(u) = f & \text{in } D, \\ u = g & \text{on } \partial D, \end{cases}$$

where $\mathcal{N}$ is a (possibly nonlinear) differential operator acting on the unknown $u : D \to \mathbb{R}$ and $f : D \to \mathbb{R}$ is a given source term. In a PINN, the continuous problem is replaced by a loss minimization problem for a neural approximation u_θ. When $\mathcal{L}$ is linear, we write $\mathcal{L}u$ instead of $\mathcal{L}(u)$, following matrix–vector notation.

Let $\{\boldsymbol{x}_i\}_{i=1}^{n_r} \subset D$ denote the interior collocation points and $\{\boldsymbol{z}_i\}_{i=1}^{n_b} \subset \partial D$ the boundary collocation points. The loss function approximating the L^2 norms on D and ∂D reads

$$J(\boldsymbol{\theta}) = \frac{1}{n_r} \sum_{i=1}^{n_r} \left|\mathcal{N}(u_\theta)(\boldsymbol{x}_i) - f(\boldsymbol{x}_i)\right|^2 + \frac{\gamma}{n_b} \sum_{i=1}^{n_b} \left|u_\theta(\boldsymbol{z}_i) - g(\boldsymbol{z}_i)\right|^2, \tag{1.1}$$

which can be interpreted as a least-squares *collocation* discretization of the PDE residual and the boundary mismatch. The parameter $\gamma > 0$ controls the balance between interior and boundary terms.

PINNs also extend to time-dependent PDEs, as we will see later through a few illustrative examples in one space dimension; see Example 1.2 and Example 1.3.

Two principal formulations are used in practice:

- **Soft enforcement (Raissi et al., 2019):** Boundary and PDE residuals are penalized jointly as in Eq. (1.1). This method is simple but introduces an additional hyperparameter γ and can suffer from poor scaling if γ is not tuned properly.

- **Hard enforcement (Lagaris et al., 1998):** The network is constructed to satisfy the boundary conditions exactly:

$$u_\theta(\boldsymbol{x}) = g_D(\boldsymbol{x}) + m(\boldsymbol{x})v_\theta(\boldsymbol{x}), \qquad \text{with } m|_{\partial D} = 0.$$

  Here g_D is a lifting of the boundary data g to D, and m a mask function vanishing on the boundary ∂D. In this case, only the interior residual term remains in the loss $J(\boldsymbol{\theta})$, removing γ and improving numerical stability and convergence.

In practice: soft enforcement is useful when boundary data are noisy/incomplete; hard enforcement is preferred when g is known exactly, as it simplifies training and avoids tuning the hyperparameter γ.

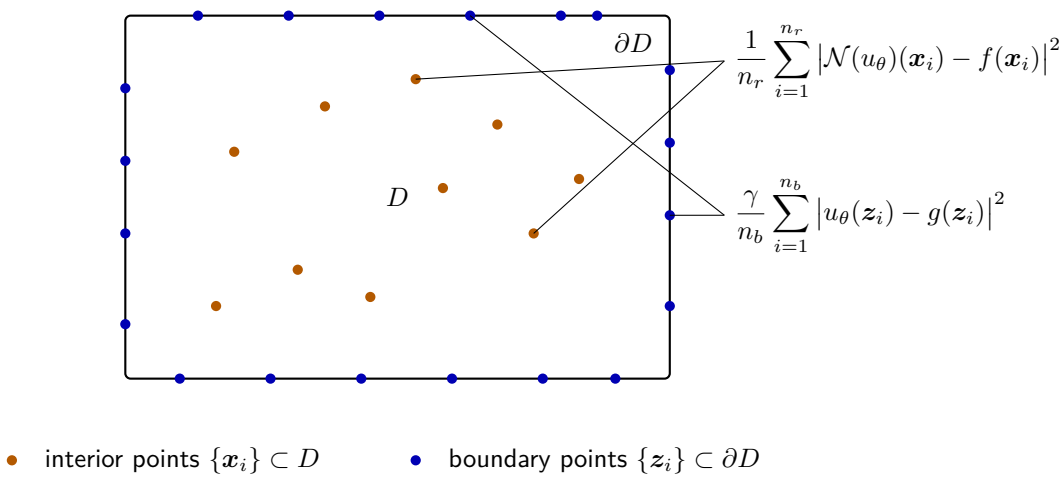


Figure 2: Interior collocation points (orange) contribute to the PDE residual sum, while boundary collocation points (blue) contribute to the boundary mismatch sum. This makes the PINN objective a least-squares *collocation* discretization of $\mathcal{N}(u) = f$ in D with $u = g$ on ∂D.

Algorithm 1 PINNs (soft enforcement)

1: Choose a neural-network architecture U_Θ parametrized by weights $\boldsymbol{\theta} \in \mathbb{R}^p$.
2: Select n_r collocation points in the domain D and n_b boundary points on ∂D.
3: Define the loss $J(\boldsymbol{\theta})$ as the sum of residual and boundary terms (coupled with $\gamma > 0$).
4: Train the network by minimizing $J(\boldsymbol{\theta})$ via backpropagation and SGD/Adam (e.g., with fixed $h = 10^{-2}$).

Example 1.1 (1D Poisson equation)**.** Consider the 1D Poisson problem on the interval $(-1, 1)$:

$$-u'' = f, \qquad u(-1) = u(1) = 0. \tag{1.2}$$

We approximate u by a neural network u_θ with weights $\boldsymbol{\theta} \in \mathbb{R}^p$. Automatic differentiation allows us to compute the PDE residual

$$r_\theta(x) = -u_\theta''(x) - f(x), \tag{1.3}$$

exactly (up to machine precision) and efficiently. The same parameters $\boldsymbol{\theta}$ are used to evaluate both u_θ and its derivatives, but computing r_θ involves higher-order differentiation through the network. The training objective combines the physics residual with the boundary loss:

$$J(\boldsymbol{\theta}) = \frac{1}{n_r}\sum_{i=1}^{n_r}|r_\theta(x_i)|^2 + \gamma\Big(|u_\theta(-1)|^2 + |u_\theta(1)|^2\Big), \tag{1.4}$$

where $\{x_i\}_{i=1}^{n_r} \subset (-1, 1)$ are the collocation points at which the PDE residual is enforced, and $\gamma > 0$ balances the two contributions. The minimization of J is performed using gradient-based optimizers such as SGD or Adam.

We next turn to time-dependent problems. One of the appealing features of PINNs is that nothing fundamental changes: the network simply takes both space and time as inputs, and automatic differentiation provides all partial derivatives required by the governing PDE.

Example 1.2 (1D heat equation)**.** On the space-time domain $(0,T)\times(-1,1)$, consider the homogeneous heat equation

$$u_t = u_{xx}, \qquad u(t,\pm 1) = 0, \quad u(0,x) = u_0(x). \tag{1.5}$$

The neural network $u_\theta(t,x)$ now depends on both time and space variables and is trained to minimize the residual

$$r_\theta(t,x) = \partial_t u_\theta(t,x) - \partial_{xx} u_\theta(t,x) \tag{1.6}$$

via the composite loss

$$J(\boldsymbol{\theta}) = \frac{1}{n_r}\sum_{i=1}^{n_r} \left|r_\theta(t_i,x_i)\right|^2 + \gamma\left[\frac{1}{n_0}\sum_{i=1}^{n_0}\left|u_\theta(0,x_i)-u_0(x_i)\right|^2 + \frac{1}{n_b}\sum_{i=1}^{n_b}\left|u_\theta(t_i,\pm 1)\right|^2\right]. \tag{1.7}$$

The first term ensures that the PDE is satisfied throughout the domain, while the second term enforces the initial and boundary conditions.

Beyond linear diffusion, the same mechanism applies without modification to nonlinear problems.

Example 1.3 (Reaction–diffusion PDEs)**.** PINNs can handle nonlinear PDEs of the form

$$u_t = \mathcal{L}u + \mathcal{N}(u), \tag{1.8}$$

where $\mathcal{L}$ is a linear differential operator and $\mathcal{N}$ a nonlinear term, such as the Allen–Cahn equation

$$u_t = u_{xx} + u - u^3,$$

by defining the residual

$$r_\theta = \partial_t u_\theta - \mathcal{L}u_\theta - \mathcal{N}(u_\theta), \tag{1.9}$$

and adding terms enforcing the initial and boundary conditions.

Relation to classical numerical methods Conceptually, PINNs sit between collocation and variational methods. Like collocation, they enforce the PDE at discrete points; like the Ritz or Galerkin methods, they minimize an energy or residual norm. The trial functions, however, are not polynomials or basis splines but neural networks, whose parameters are optimized globally rather than locally. This allows PINNs to handle high-dimensional or irregular domains more flexibly, albeit at the cost of a nonconvex optimization problem.

Takeaways

PINNs approximate PDE solutions with neural networks by minimizing a residual-based loss. Key points:

- **Neural ansatz:** the network acts as a differentiable trial function for the solution.
- **Physics residual:** derivatives are computed via automatic differentiation; the PDE residual guides training.
- **Boundary/initial conditions:** incorporated in the loss via soft penalties or hard enforcement.
- **Flexibility:** the method extends naturally to time-dependent problems, nonlinear PDEs, higher dimensions, and irregular domains.
- **Optimization:** nonconvex loss handled via gradient-based methods (SGD, Adam).

Lesson for SciML: PINNs unify numerical analysis and machine learning, embedding physical laws directly into the learning process.

1.2 Theoretical results

Setup Let $D \subset \mathbb{R}^d$ be an open, bounded set with smooth boundary ∂D. We consider the Dirichlet problem

$$\mathcal{L}u = f \quad \text{in } D, \qquad u = g \quad \text{on } \partial D, \tag{1.10}$$

where $\mathcal{L}$ is a (possibly variable-coefficient) second-order uniformly elliptic operator acting on the unknown $u : D \to \mathbb{R}$, $f \in L^2(D)$ is a *source* (also known as *forcing*) term, and $g \in L^2(\partial D)$ is boundary data.

Under standard assumptions, Eq. (1.10) admits a unique solution u in the operator domain

$$\mathrm{dom}(\mathcal{L}) \subset H^2(D) \cap \{H^1(D) \text{ with } u|_{\partial D} = g\}.$$

We approximate u by a neural network $u_\theta \in U_\Theta$ parametrized by weights $\boldsymbol{\theta} \in \Theta \subset \mathbb{R}^p$ for some $p \geq 1$; to evaluate $\mathcal{L}u_\theta$, we assume U_Θ uses sufficiently smooth activations.

Errors We measure performance with the *generalization error*

$$\mathcal{E}_G(\theta) = \|\mathcal{L}u_\theta - f\|^2_{L^2(D)} \; + \; \gamma \, \|u_\theta - g\|^2_{L^2(\partial D)}, \qquad \gamma > 0, \tag{1.11}$$

and, in practice, minimize its *training* (collocation) counterpart

$$\mathcal{E}_T(\theta) = \frac{1}{n_r} \sum_{i=1}^{n_r} \big|\mathcal{L}u_\theta(\boldsymbol{x}_i) - f(\boldsymbol{x}_i)\big|^2 + \frac{\gamma}{n_b} \sum_{i=1}^{n_b} \big|u_\theta(\boldsymbol{z}_i) - g(\boldsymbol{z}_i)\big|^2, \tag{1.12}$$

where $\{\boldsymbol{x}_i\}_{i=1}^{n_r} \subset D$ are interior (residual) points and $\{\boldsymbol{z}_i\}_{i=1}^{n_b} \subset \partial D$ are boundary points. Let

$$\boldsymbol{\theta}^*_G \in \arg\min_{\boldsymbol{\theta} \in \Theta} \mathcal{E}_G(\boldsymbol{\theta}), \qquad \boldsymbol{\theta}^*_T \in \arg\min_{\boldsymbol{\theta} \in \Theta} \mathcal{E}_T(\boldsymbol{\theta}).$$

Then, as in standard learning decomposition, for any $\boldsymbol{\theta} \in \Theta$,

$$\mathcal{E}_G(\boldsymbol{\theta}) \; \leq \; \mathcal{E}_G(\boldsymbol{\theta}^*_G) \; + \; 2 \sup_{\boldsymbol{\theta}' \in \Theta} \big|\mathcal{E}_G(\boldsymbol{\theta}') - \mathcal{E}_T(\boldsymbol{\theta}')\big| \; + \; \mathcal{E}_T(\boldsymbol{\theta}) - \mathcal{E}_T(\boldsymbol{\theta}^*_T). \tag{1.13}$$

What ultimately matters is the *solution error*

$$\mathcal{E}(\boldsymbol{\theta}) := \|u_\theta - u\|^2_{H^2(D)}. \tag{1.14}$$

Stability Assuming the inverse $\mathcal{L}^{-1} \colon L^2(D) \to \mathrm{dom}(\mathcal{L})$ is bounded, we have for all $v, w \in \mathrm{dom}(\mathcal{L})$,

$$\|v - w\|_{H^2(D)} = \|\mathcal{L}^{-1}(\mathcal{L}v - \mathcal{L}w)\|_{H^2(D)} \; \leq \; C \, \|\mathcal{L}v - \mathcal{L}w\|_{L^2(D)}. \tag{1.15}$$

With $w = u$ and $v = u_\theta$, this yields

$$\mathcal{E}(\boldsymbol{\theta}) \; \leq \; C \, \|\mathcal{L}u_\theta - f\|^2_{L^2(D)} \; \leq \; C \, \mathcal{E}_G(\boldsymbol{\theta}), \tag{1.16}$$

so controlling $\mathcal{E}_G$ controls the solution error.

Theorem 1.4 (Errors of PINNs). *For any $\boldsymbol{\theta} \in \Theta$,*

$$\underbrace{\mathcal{E}(\boldsymbol{\theta})}_{\textit{solution error}} \; \leq \; C \; \underbrace{\mathcal{E}_G(\boldsymbol{\theta})}_{\textit{generalization error}} \; \leq \; C \Bigg[\underbrace{\mathcal{E}_G(\boldsymbol{\theta}^*_G)}_{\textit{approximation error}} + \underbrace{2 \sup_{\boldsymbol{\theta}' \in \Theta} |\mathcal{E}_G(\boldsymbol{\theta}') - \mathcal{E}_T(\boldsymbol{\theta}')|}_{\textit{generalization gap}} + \underbrace{\mathcal{E}_T(\boldsymbol{\theta}) - \mathcal{E}_T(\boldsymbol{\theta}^*_T)}_{\textit{optimization error}} \Bigg].$$

Theorem 1.4 mirrors classical stability results in numerical analysis, where one typically invokes the principle "consistency + stability implies convergence." Here, the generalization gap plays the role of consistency, the constant C quantifies stability, and an additional term accounts for errors arising from imperfect optimization.

Approximation error and its interpretation The approximation error corresponds to the first term in the decomposition of Theorem 1.4 and measures how well the neural-network space U_Θ can represent the true solution u of Eq. (1.10). For illustration, consider the Poisson operator $\mathcal{L} = \Delta$. Let $u^*_G := u_{\boldsymbol{\theta}^*_G}$. Then

$$\|\mathcal{L}u^*_G - f\|_{L^2(D)} = \|\Delta(u^*_G - u)\|_{L^2(D)} \; \leq \; \|u^*_G - u\|_{H^2(D)}. \tag{1.17}$$

Using the continuity of the (trace) operator $u \in H^2(D) \mapsto u|_{\partial D} \in L^2(\partial D)$, this gives

$$\mathcal{E}_G(u_G^*) = \|\mathcal{L}u_G^* - f\|_{L^2(D)}^2 + \gamma \|u_G^* - g\|_{L^2(\partial D)}^2 \;\leq\; (1+\gamma C)\, \|u_G^* - u\|_{H^2(D)}^2. \tag{1.18}$$

Hence, controlling the generalization error reduces to controlling the approximation $\|u_G^* - u\|_{H^2(D)}$ of the target solution by the neural network space.

Current approximation theory for neural networks provides such guarantees in weaker norms. For example, Siegel [13, Thm. 1] proved that if $u \in H^2(D)$, then for any $\varepsilon > 0$ there exists a ReLU network $u_\theta \in U_\Theta$ of size $\mathcal{O}(\varepsilon^{-d/4})$ such that

$$\|u - u_\theta\|_{L^2(D)} \leq \varepsilon, \tag{1.19}$$

but this does not extend directly to the $H^2(D)$-norm in Eq. (1.18). Indeed, a network can approximate u accurately in L^2 while still producing oscillatory derivatives, resulting in a potentially large H^2-norm.

For smoother activation functions and under additional regularity assumptions on u, stronger results exist: it is possible to bound $\|u - u_\theta\|_{H^2(D)}$ in terms of $\|u - u_\theta\|_{L^2(D)}$ and higher-order derivatives of the network; see, e.g., De Ryck and Mishra [4, Thm. 4.11]. These results demonstrate that the *approximation error can, in principle, be controlled theoretically*, although quantitative H^2-rates remain an open area of PINN analysis.

Generalization gap and its control The generalization gap measures the difference

$$|\mathcal{E}_G(\boldsymbol{\theta}) - \mathcal{E}_T(\boldsymbol{\theta})|,$$

reflecting how well the empirical collocation loss approximates the PDE and boundary losses. It arises purely from *quadrature error*: $\mathcal{E}_T$ replaces the integrals in $\mathcal{E}_G$ by finite sums over interior and boundary points.

For example, in one space dimension,

$$I = \int_{-1}^{1} f(x)\, dx, \qquad I_n = \sum_{i=1}^{n} w_i f(x_i),$$

with $\{x_i, w_i\}$ quadrature nodes and weights. If $f \in C^2([-1,1])$, the midpoint rule with spacing $h = 2/n$ gives $|I - I_n| \leq Cn^{-2}$, while higher-order rules (Gauss, Clenshaw–Curtis) yield n^{-k} for C^k functions.

In d dimensions, midpoint converges as $n^{-2/d}$, Gauss/Clenshaw–Curtis as $n^{-k/d}$, and Monte–Carlo as $n^{-1/2}$, independent of d. Since the PINN loss $\mathcal{E}_T$ is a quadrature approximation of $\mathcal{E}_G$, classical results imply

$$\sup_{\boldsymbol{\theta}\in\Theta} |\mathcal{E}_G(\boldsymbol{\theta}) - \mathcal{E}_T(\boldsymbol{\theta})| \xrightarrow[n_r, n_b \to \infty]{} 0,$$

with rates dictated by the regularity of $\mathcal{L}u_\theta - f$ and the sampling scheme. Hence, increasing collocation points or using higher-order/randomized quadrature allows the *generalization gap to be made arbitrarily small*.

Optimization error: the main bottleneck The last term in Theorem 1.4 is the *optimization error*,

$$\mathcal{E}_T(\boldsymbol{\theta}) - \mathcal{E}_T(\boldsymbol{\theta}_T^*),$$

which quantifies how well the training procedure minimizes the empirical loss. Unlike approximation and quadrature errors, which can be theoretically controlled, the optimization error remains the main practical challenge in PINNs. The loss $\mathcal{E}_T(\boldsymbol{\theta})$ is highly nonconvex and its gradient is generally non-Lipschitz, so standard convergence guarantees for gradient descent (see Chapter 2) do not directly apply.

Takeaways

- **Approximation error:** reflects the expressive power of the neural network class U_Θ; controlling $\|u - u_G^*\|_{H^2(D)}$ is key to bounding the PDE residual.
- **Generalization gap:** arises from evaluating the loss at discrete collocation points; standard quadrature results allow theoretical control of this term.
- **Optimization error:** corresponds to the practical error from minimizing a nonconvex loss; remains the main practical challenge in PINNs.

2 Training dynamics of PINNs

Observations from the previous section highlight the role of optimization, motivating an analysis of PINN training dynamics via linearized flows and Neural Tangent Kernel theory.

2.1 Linearized dynamics

Partial theoretical insight can be gained by studying simplified models of the training dynamics. A classical approach is to linearize the PINN loss around the initialization, as proposed in [3].

Loss Consider the following continuous loss for the Dirichlet problem of Section 1.2:

$$\mathcal{J}(\boldsymbol{\theta}) = \frac{1}{2}\|\mathcal{L}u_\theta - f\|^2_{L^2(D)} + \frac{\gamma}{2}\|u_\theta - g\|^2_{L^2(\partial D)}.$$

Training dynamics Consider the gradient-descent iteration

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^k - h\nabla\mathcal{J}(\boldsymbol{\theta}^k), \quad h > 0.$$

Regime Assume that u_θ behaves approximately linearly in $\boldsymbol{\theta}$ near $\boldsymbol{\theta}^0$, which yields

$$u_{\theta^k} \approx u_{\theta^0} + (\nabla_\theta u_{\theta^0})^\top(\theta^k - \theta^0).$$

Simplified dynamics Setting $\phi_i := \partial_{\theta_i} u_{\theta^0}$, the training dynamics reduce to a linear system involving the following matrix $A \in \mathbb{R}^{p\times p}$ and vector $\boldsymbol{b} \in \mathbb{R}^p$:

$$A_{ij} = (\mathcal{L}\phi_i, \mathcal{L}\phi_j)_{L^2(D)} + \gamma(\phi_i, \phi_j)_{L^2(\partial D)}, \quad b_i = (f - \mathcal{L}u_{\theta^0}, \mathcal{L}\phi_i)_{L^2(D)} + \gamma(g - u_{\theta^0}, \phi_i)_{L^2(\partial D)}.$$

The corresponding iteration is a linearized gradient descent:

$$\boldsymbol{\theta}^{k+1} = (I - hA)\boldsymbol{\theta}^k + h(A\boldsymbol{\theta}^0 + \boldsymbol{b}).$$

Since A is symmetric (PSD in general, assumed SPD here), all eigenvalues are real and positive. Let $\lambda_j(A)$ denote the eigenvalues of A, and define $\lambda_{\min}(A) = \min_j \lambda_j(A)$, $\lambda_{\max}(A) = \max_j \lambda_j(A)$. The spectrum $0 < \lambda_{\min}(A) \le \lambda_{\max}(A)$ controls the conditioning of the linearized problem.

Theorem 2.1 (Convergence of linearized gradient descent)**.** *For any step size $h \in (0, 1/\lambda_{\max}(A)]$, the linearized gradient-descent sequence converges to $\boldsymbol{\theta}^* = \boldsymbol{\theta}^0 + A^{-1}\boldsymbol{b}$ linearly:*

$$\|\boldsymbol{\theta}^k - \boldsymbol{\theta}^*\|_2 \le (1 - h\lambda_{\min}(A))^k\|\boldsymbol{\theta}^0 - \boldsymbol{\theta}^*\|_2.$$

Proof

We have

$$\boldsymbol{\theta}^k = (I - hA)\boldsymbol{\theta}^{k-1} + h(A\boldsymbol{\theta}^0 + \boldsymbol{b}) = (I - hA)^k\boldsymbol{\theta}^0 + \sum_{\ell=0}^{k-1}(I - hA)^\ell h(A\boldsymbol{\theta}^0 + \boldsymbol{b}),$$

which yields

$$\boldsymbol{\theta}^k = (I - hA)^k\boldsymbol{\theta}^0 + (hA)^{-1}(I - (I - hA)^k)h(A\boldsymbol{\theta}^0 + \boldsymbol{b}) = (I - hA)^k\boldsymbol{\theta}^0 + (I - (I - hA)^k)\boldsymbol{\theta}^*.$$

We can rewrite this as $\boldsymbol{\theta}^k - \boldsymbol{\theta}^* = (I - hA)^k(\boldsymbol{\theta}^0 - \boldsymbol{\theta}^*)$. This leads to

$$\begin{aligned}\|\boldsymbol{\theta}^k - \boldsymbol{\theta}^*\|_2 &\le \|I - hA\|_2^k\|\boldsymbol{\theta}^0 - \boldsymbol{\theta}^*\|_2 = \max_j |1 - h\lambda_j(A)|^k\|\boldsymbol{\theta}^0 - \boldsymbol{\theta}^*\|_2,\\ &\le |1 - h\lambda_{\min}(A)|^k\|\boldsymbol{\theta}^0 - \boldsymbol{\theta}^*\|_2.\end{aligned}$$

For $h \in (0, 1/\lambda_{\max}(A)]$ and $\lambda_{\min}(A) > 0$, $(1 - h\lambda_{\min}(A)) \in [0, 1)$. This concludes the proof.

Consequences Define the condition number of A as

$$\kappa(A) = \lambda_{\max}(A)/\lambda_{\min}(A) \ge 1. \tag{2.1}$$

Then for $h = 1/\lambda_{\max}(A)$, we obtain convergence at the rate

$$(1 - \kappa(A)^{-1})^k. \tag{2.2}$$

The parameter γ plays a crucial role as it balances the relative contributions of the physics-informed cost function. Given the analysis of the previous sections, it is natural to suggest

$$\gamma^* = \min_{\gamma} \kappa(A(\gamma)) \tag{2.3}$$

in order to obtain the smallest condition number and accelerate convergence. To further improve this rate, one can try to decrease the condition number with techniques called preconditioning, where instead of solving a linear system of the form $A\boldsymbol{x} = \boldsymbol{b}$ one solves $M^{-1}A\boldsymbol{x} = M^{-1}A\boldsymbol{b}$, where M is constructed so that $\kappa(M^{-1}A) \ll \kappa(A)$ and $M\boldsymbol{x} = \boldsymbol{c}$ can be solved fast. Similar ideas can be applied here [3].

2.2 Neural Tangent Kernel theory

The *Neural Tangent Kernel* (NTK) framework provides another analytical tool to study PINN training dynamics. It formalizes the linearization of the network output in the infinite-width limit, allowing the evolution of u_θ during gradient descent to be expressed as a linear dynamical system.

Loss Consider the following discrete loss for the Dirichlet problem of Section 1.2, with $\gamma = 1$ for now:

$$J(\boldsymbol{\theta}) = \frac{1}{2}\sum_{i=1}^{n_r} \left|\mathcal{L}u_\theta(\boldsymbol{x}_i) - f(\boldsymbol{x}_i)\right|^2 + \frac{1}{2}\sum_{i=1}^{n_b} \left|u_\theta(\boldsymbol{z}_i) - g(\boldsymbol{z}_i)\right|^2.$$

Our analysis will help us to choose γ to balance out terms "correctly."

Training dynamics Consider the continuous-time limit of gradient descent as $h \to 0$ (gradient flow):

$$\dot{\boldsymbol{\theta}}(t) = -\nabla_\theta J(\boldsymbol{\theta}(t)).$$

Let $\boldsymbol{e}_b(t)$ and $\boldsymbol{e}_r(t)$ denote the vectors of boundary errors $u_{\theta(t)}(\boldsymbol{z}_i) - g(\boldsymbol{z}_i)$ and PDE residuals $\mathcal{L}u_{\theta(t)}(\boldsymbol{x}_i) - f(\boldsymbol{x}_i)$. Then from

$$\frac{d}{dt}\mathcal{L}u_{\theta(t)}(\boldsymbol{x}_i) = \nabla_\theta \mathcal{L}u_{\theta(t)}(\boldsymbol{x}_i)^\top \dot{\boldsymbol{\theta}}(t), \quad \frac{d}{dt}u_{\theta(t)}(\boldsymbol{z}_i) = \nabla_\theta u_{\theta(t)}(\boldsymbol{z}_i)^\top \dot{\boldsymbol{\theta}}(t),$$

we obtain

$$\frac{d}{dt}\begin{bmatrix} \boldsymbol{e}_b(t) \\ \boldsymbol{e}_r(t) \end{bmatrix} = -\underbrace{\begin{bmatrix} K_{bb}(t) & K_{br}(t) \\ K_{rb}(t) & K_{rr}(t) \end{bmatrix}}_{K(t)} \begin{bmatrix} \boldsymbol{e}_b(t) \\ \boldsymbol{e}_r(t) \end{bmatrix}, \tag{2.4}$$

where the NTK blocks are defined by

$$\begin{aligned} (K_{bb}(t))_{ij} &= \nabla_\theta u_{\theta(t)}(\boldsymbol{z}_i)^\top \nabla_\theta u_{\theta(t)}(\boldsymbol{z}_j), & (K_{rr}(t))_{ij} &= \nabla_\theta \mathcal{L}u_{\theta(t)}(\boldsymbol{x}_i)^\top \nabla_\theta \mathcal{L}u_{\theta(t)}(\boldsymbol{x}_j), \\ (K_{br}(t))_{ij} &= \nabla_\theta u_{\theta(t)}(\boldsymbol{z}_i)^\top \nabla_\theta \mathcal{L}u_{\theta(t)}(\boldsymbol{x}_j). & K_{rb}(t) &= K_{br}(t)^\top. \end{aligned}$$

Regime At initialization, the NTK of a one-hidden-layer PINN satisfies $K(0) \to K$ as the width $N \to \infty$, where K is a deterministic infinite-width kernel. Moreover, during training the NTK remains essentially constant: $K(t) \approx K(0)$ for all t. Hence, in the wide-network limit, the PINN operates in the kernel regime

$$K(t) \;\approx\; K(0) \;\approx\; K.$$

Thus, the network approximately evolves according to a fixed, PSD kernel operator K, transforming the originally nonconvex optimization problem into a linear dynamical system whose convergence can be analyzed through K.

Simplified dynamics Let $\boldsymbol{e} = (\boldsymbol{e}_b, \boldsymbol{e}_r)^\top$. With constant K, the dynamics reads

$$\frac{d}{dt}\boldsymbol{e}(t) = -K\boldsymbol{e}(t) \qquad \Longrightarrow \qquad \boldsymbol{e}(t) = \exp(-Kt)\boldsymbol{e}(0).$$

Let $Q\Lambda Q^\top$ be the eigendecomposition of K and write $\boldsymbol{e} = (\boldsymbol{e}_b, \boldsymbol{e}_r)^\top$. Then

$$Q^\top \boldsymbol{e}(t) = \exp(-\Lambda t) Q^\top \boldsymbol{e}(0).$$

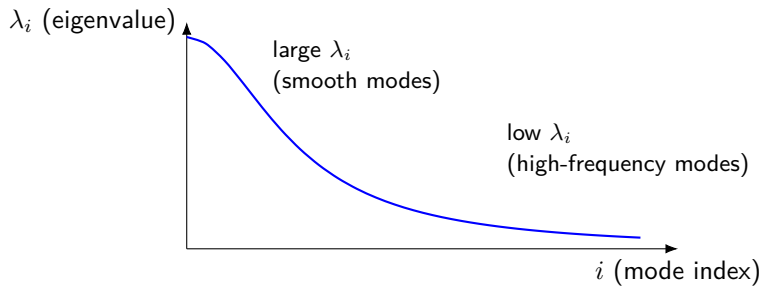


Figure 3: Typical decay of NTK eigenvalues.

Hence, modes associated with large λ_i decay quickly, whereas those with small λ_i converge slowly. In PINNs, the spectrum of K typically decays rapidly, implying that:

- high-frequency (oscillatory) components typically associated with small eigenvalues λ_i are learned slowly, a manifestation of *spectral bias*;
- the resulting scale separation between large and small eigenvalues induces numerical stiffness, forcing the use of very small learning rates at the discrete level.

A schematic representation of this effect is shown in Fig. 3.

Consequences The NTK matrix in Eq. (2.4) consists of four blocks coupling the PDE residual and boundary terms. Wang *et al.* [15] proposed to balance their contributions by rescaling the loss weights according to

$$\gamma_b = \frac{\mathrm{Tr}(K_{bb}) + \mathrm{Tr}(K_{rr})}{\mathrm{Tr}(K_{bb})}, \qquad \gamma_r = \frac{\mathrm{Tr}(K_{bb}) + \mathrm{Tr}(K_{rr})}{\mathrm{Tr}(K_{rr})}. \tag{2.5}$$

Takeaways

The main analytical difficulty in PINNs is the *optimization error*. Two partial analyses are useful:

- **Linearized dynamics:** captures local behavior around initialization; convergence governed by the spectrum of the Jacobian A.
- **NTK:** dynamics driven by the kernel K; eigenmodes decay at rates λ_i, causing spectral bias and stiffness between boundary and residual blocks.

	Linearized dynamics	**NTK**
Loss	continuous $\mathcal{J}(\boldsymbol{\theta})$	discrete $J(\boldsymbol{\theta})$
Training dynamics	$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^k - h\nabla\mathcal{J}(\boldsymbol{\theta}^k)$	$\dot{\boldsymbol{\theta}} = -\nabla J(\boldsymbol{\theta})$
Regime	u_θ linear in $\boldsymbol{\theta}$ around $\boldsymbol{\theta}^0$	infinite-width, single-layer u_θ
Simplified dynamics	$\boldsymbol{\theta}^{k+1} = (I - hA)\boldsymbol{\theta}^k + h(A\boldsymbol{\theta}^0 + \boldsymbol{b})$	$\dot{\boldsymbol{e}}(t) = -K\boldsymbol{e}(t)$
Solution	$\boldsymbol{\theta} = \boldsymbol{\theta}^0 + A^{-1}\boldsymbol{b}$	$\boldsymbol{e}(t) = \exp(-Kt)\boldsymbol{e}(0)$

2.3 Other practical considerations

Optimization algorithms PINN training is typically performed with the Adam optimizer using learning rates such as $h = 10^{-2}$ or 10^{-3}, sometimes with learning-rate decay. A common strategy is to first minimize the loss with Adam for a fixed number of iterations, then refine the solution with L-BFGS (limited-memory BFGS).

Network architecture PINNs commonly use fully connected networks with the $\tanh$ activation, though other activations (e.g., sigmoid) and architectures (e.g., convolutional or recurrent networks) have also been explored. More recently, Kolmogorov–Arnold networks have been proposed as an alternative. See [2] for a review of architectures used in PINNs.

3 The Deep Ritz method

In what follows we illustrate an alternative approach based on a variational principle rather than the strong form of a PDE. Consider the 1D Poisson problem

$$-u''(x) = f(x), \qquad u(0) = u(1) = 0.$$

The classical Dirichlet energy functional is

$$\mathcal{J}(u) := \int_0^1 \mathcal{L}(x, u, u')dx, \qquad \mathcal{L}(x, u, u') = \tfrac{1}{2}|u'(x)|^2 - f(x)u(x), \tag{3.1}$$

whose Euler–Lagrange equation

$$\frac{\partial \mathcal{L}}{\partial u} - \frac{d}{dx}\frac{\partial \mathcal{L}}{\partial u'} = -f - u'' = 0$$

recovers $-u'' = f$ with the given boundary conditions.

Function-space gradient flow Let u^* be the unique minimizer of $\mathcal{J}$ (the weak solution of $-u'' = f$ with $u(0) = u(1) = 0$). The L^2-gradient of $\mathcal{J}$ is

$$\nabla_u \mathcal{J}(u) = -u'' - f.$$

The (ideal) L^2-*gradient flow* evolves the function in fictitious optimization time t:

$$\begin{aligned} &\partial_t u(t, x) = -\nabla_u \mathcal{J}(u(t, \cdot)) = u''(t, x) + f(x), \\ &u(t, 0) = u(t, 1) = 0. \end{aligned}$$

Subtracting the steady state u^* shows

$$\partial_t(u(t) - u^*) = (u(t) - u^*)'',$$

which is a heat equation. Expanding in Dirichlet Laplacian eigenpairs $\{\lambda_k, \varphi_k\}$, $\lambda_k = (k\pi)^2$, gives

$$u(t) - u^* = \sum_{k=1}^{\infty} c_k(0)\, e^{-\lambda_k t}\, \varphi_k, \quad \Rightarrow \quad \|u(t) - u^*\|_{L^2(0,1)} \le e^{-\lambda_1 t}\, \|u(0) - u^*\|_{L^2(0,1)}.$$

Moreover, the energy decreases monotonically:

$$\frac{d}{dt}\mathcal{J}(u(t)) = (\nabla_u \mathcal{J}(u(t)), \partial_t u(t))_{L^2(0,1)} = -\|\nabla_u \mathcal{J}(u(t))\|^2_{L^2(0,1)} \le 0.$$

Thus, at the *function level*, gradient flow is parabolic and well-conditioned, with spectral gap $\lambda_1 = \pi^2$ in 1D.

Neural network approximation (Deep Ritz) We discretize the function space using a neural network with a *mask* $m(x)$ that enforces the boundary conditions:

$$u_\theta(x) = m(x)v_\theta(x), \qquad m(0) = m(1) = 0 \quad \text{(e.g., } m(x) = x(1-x)\text{)}.$$

Then $u_\theta(0) = u_\theta(1) = 0$ exactly.

Discrete (collocation/quadrature) loss Let $\{x_i\}_{i=1}^{n_r} \subset (0, 1)$. Then the discrete loss reads

$$J(\boldsymbol{\theta}) = \frac{1}{n_r}\sum_{i=1}^{n_r}\left(\tfrac{1}{2}\,|\partial_x u_\theta(x_i)|^2 - f(x_i)u_\theta(x_i)\right).$$

Algorithm 2 Deep Ritz (1D Poisson with hard Dirichlet BC)

1: Choose a mask function m and set $u_\theta = mv_\theta$.
2: Choose a neural-network architecture V_Θ for v_θ parametrized by weights $\boldsymbol{\theta} \in \mathbb{R}^p$.
3: Select n_r collocation points in the domain.
4: Define the loss $J(\boldsymbol{\theta})$.
5: Train the network by minimizing $J(\boldsymbol{\theta})$ via backpropagation and SGD/Adam (e.g., with fixed $h = 10^{-2}$).

Linearization We can also perform linearization as before to analyze the training dynamics. Consider the following continuous loss

$$\mathcal{J}(\boldsymbol{\theta}) = \tfrac{1}{2}\|\partial_x u_\theta\|^2_{L^2(0,1)} - (f, u_\theta)_{L^2(0,1)}.$$

Then, from

$$\partial_{\theta_j}\mathcal{J}(\boldsymbol{\theta}) = (\partial_x u_\theta, \partial_{\theta_j}\partial_x u_\theta)_{L^2(0,1)} - (f, \partial_{\theta_j} u_\theta)_{L^2(0,1)},$$

and by linearizing around $\boldsymbol{\theta}^0$ and setting $\phi_i := \partial_{\theta_i} u_{\theta^0}$, we obtain

$$\boldsymbol{\theta}^{k+1} = (I - hA)\boldsymbol{\theta}^k + h(A\boldsymbol{\theta}^0 + \boldsymbol{b}),$$

with

$$A_{ij} = (\partial_x\phi_i, \partial_x\phi_j)_{L^2(0,1)}, \quad b_i = (f, \phi_i)_{L^2(0,1)} - (\partial_x u_{\theta^0}, \partial_x\phi_i)_{L^2(D)}.$$

Because the matrix A involves lower derivative terms compared to those obtained with PINNs, we expect the condition number to be more favorable and convergence to be faster.

What we can prove (and why it helps)

- *Function-space guarantees:* For the Dirichlet energy in Eq. (3.1), the L^2-gradient flow decreases $\mathcal{J}$ monotonically and converges exponentially to u^* with rate λ_1.
- *Parameter-space insights:* Linearized dynamics for Deep Ritz inherit this favorable spectrum: the problem is typically better conditioned than the *second-order* squared residual in PINNs. This reduces stiffness and sensitivity to loss balancing.

Beyond 1D In multiple dimensions, for $-\Delta u = f$ in D with $u|_{\partial D} = 0$, the Deep Ritz functional is

$$J(u) = \int_D \left(\tfrac{1}{2}|\nabla u|^2 - f\,u\right) dx,$$

and the same principles apply: hard BC via $u_\theta = m v_\theta$, quadrature-based training, monotone energy decay for function-space gradient flow, and improved conditioning relative to strong-residual PINNs. For Neumann/Robin BCs, modify the energy or incorporate boundary terms accordingly.

Takeaways

- **Loss structure:** PINNs minimize the strong residual; Deep Ritz minimizes a convex energy whenever a variational form exists.
- **Conditioning:** Function-space gradient flow for Deep Ritz is parabolic with rate λ_1 (smoothing). PINNs' residual is typically squared (higher-order, stiffer).
- **Boundary conditions:** Deep Ritz naturally uses *hard* Dirichlet BC; no loss-weight tuning is needed.
- **Theory:** Energy decreases monotonically; convergence to u^* follows from spectral gap at the function level. In parameter space, linearized analysis carries over with better spectral properties than strong-residual PINNs.
- **Scope:** Deep Ritz requires a variational (energy) formulation. For PDEs where no such principle holds (e.g., advection–reaction), one must use mixed/weak variants (Deep Galerkin, VPINNs, mixed Ritz).

4 Automatic differentiation

Forward mode To compute derivatives, one could use finite differences, e.g.,

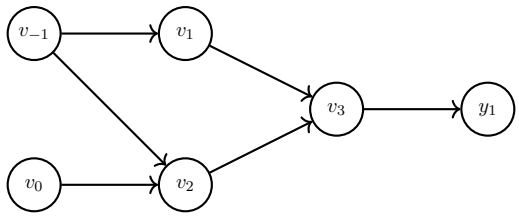
$$u'(x) \approx \frac{u(x+h)-u(x)}{h} + \mathcal{O}(h) \qquad \text{or} \qquad u'(x) \approx \frac{u(x+h)-u(x-h)}{2h} + \mathcal{O}(h^2). \tag{4.1}$$

However, it introduces an error that needs to be tracked down and adjusted, and a prohibitive cost in high dimensions. For a function $u : \mathbb{R}^d \to \mathbb{R}^{d'}$, computing the Jacobian matrix requires $\mathcal{O}(dd')$ evaluations.

This where *automatic differentiation* (AD) comes in. Suppose $u : \mathbb{R}^d \to \mathbb{R}^{d'}$ is constructed using intermediate variables v_i such that v_{i-d}, $1 \le i \le d$, are the input variables, v_i are the working variables, and $y_{d'-i}$, $0 \le i \le d'-1$ are the output variables.

For instance, consider $u : \mathbb{R}^2 \to \mathbb{R}$, $u(x_1, x_2) = \sin x_1 - x_1 x_2$ with $d = 2$ and $d' = 1$. The variables are

$$v_{-1} = x_1, \quad v_0 = x_2, \quad v_1 = \sin v_{-1}, \quad v_2 = v_{-1} \times v_0, \quad v_3 = v_1 - v_2, \quad y_1 = v_3. \tag{4.2}$$



To compute derivatives with respect to, e.g., x_1, we start by associating with each variable v_i a derivative $\dot{v}_i = \partial_{x_1} v_i$, with $\dot{v}_{-1} = \dot{x}_1 = 1$ and $\dot{v}_0 = \dot{x}_2 = 0$. Applying the chain rule to each elementary operation in the *forward primal trace*, we generate the corresponding *tangent derivative trace*. For instance, in our case,

$$\dot{v}_1 = \dot{v}_{-1} \times \cos v_{-1}, \quad \dot{v}_2 = \dot{v}_{-1} \times v_0 + v_{-1} \times \dot{v}_0, \quad \dot{v}_3 = \dot{v}_1 - \dot{v}_2, \quad \dot{y}_1 = \dot{v}_3. \tag{4.3}$$

Here is an example to compute our function and its derivative with respect to x_1 at the point $(2, 3)$.

Forward Primal Trace			Forward Derivative Trace (↓)		
v_{-1}	$= x_1$	$= 2$	$\dot{v}_{-1}$	$= \dot{x}_1$	$= 1$
v_0	$= x_2$	$= 3$	$\dot{v}_0$	$= \dot{x}_2$	$= 0$
v_1	$= \sin v_{-1}$	$= \sin 2$	$\dot{v}_1$	$= \dot{v}_{-1} \times \cos v_{-1}$	$= 1 \times \cos 2$
v_2	$= v_{-1} \times v_0$	$= 2 \times 3$	$\dot{v}_2$	$= \dot{v}_{-1} \times v_0 + v_{-1} \times \dot{v}_0$	$= 1 \times 3 + 2 \times 0$
v_3	$= v_1 - v_2$	$= \sin 2 - 6$	$\dot{v}_3$	$= \dot{v}_1 - \dot{v}_2$	$= \cos 2 - 3$
y_1	$= v_3$	$= \sin 2 - 6$	$\dot{y}_1$	$= \dot{v}_3$	$= \cos 2 - 3$

This generalizes to computing the Jacobian of a function $u : \mathbb{R}^d \to \mathbb{R}^{d'}$ with d input variables x_i and d' output variables y_j. In this case, each forward pass is initialized by setting only one of the variables $\dot{x}_i = 1$ and setting the rest to zero; in other words, setting $\dot{\boldsymbol{x}} = \boldsymbol{e}_i$ where $\boldsymbol{e}_i$ is the i-th unit vector. A run of the code with $\boldsymbol{x} = \boldsymbol{a}$ computes

$$\dot{y}_j = \frac{\partial y_j}{\partial x_i}(\boldsymbol{a}), \quad j = 1, \ldots, d', \tag{4.4}$$

giving us one column of the Jacobian matrix

$$J = \begin{bmatrix} \frac{\partial y_1}{\partial x_1}(\boldsymbol{a}) & \ldots & \frac{\partial y_1}{\partial x_d}(\boldsymbol{a}) \\ \vdots & & \vdots \\ \frac{\partial y_{d'}}{\partial x_1}(\boldsymbol{a}) & \ldots & \frac{\partial y_{d'}}{\partial x_d}(\boldsymbol{a}) \end{bmatrix} \in \mathbb{R}^{d' \times d}. \tag{4.5}$$

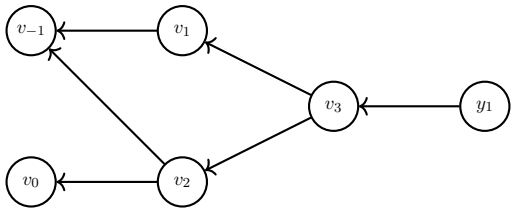
Therefore, computing the full Jacobian requires d forward passes. However, computing matrix-vector products $J\boldsymbol{r}$ for some vector $\boldsymbol{r}$ can be done with only one pass by setting $\dot{\boldsymbol{x}} = \boldsymbol{r}$.

Reverse mode Instead of propagating derivatives forward, reverse mode computes derivatives *backward*, accumulating sensitivities from the outputs to the inputs. This *reverse* AD mode corresponds to a generalized backpropagation algorithm [1].

Consider our previous example with $u(x_1, x_2) = \sin x_1 - x_1 x_2$. To compute derivatives with respect to x_1, we define the *adjoint variables* $\bar{v}_i = \partial y_1 / \partial v_i$:

$$\bar{v}_3 = \bar{y}_1, \quad \bar{v}_2 = \bar{v}_3 \times \frac{\partial v_3}{\partial v_2}, \quad \bar{v}_1 = \bar{v}_3 \times \frac{\partial v_3}{\partial v_1}, \quad \bar{v}_0 = \bar{v}_2 \times \frac{\partial v_2}{\partial v_0}, \quad \bar{v}_{-1} = \bar{v}_1 \times \frac{\partial v_1}{\partial v_{-1}} + \bar{v}_2 \times \frac{\partial v_2}{\partial v_{-1}}.$$

Let us carry out that computation at the point $(2, 3)$. The adjoint variables in the table are evaluated in a top-to-bottom (reverse-mode) order.



Forward Primal Trace			Reverse Adjoint Trace (↑)		
v_{-1}	$= x_1$	$= 2$	$\bar{v}_{-1}$	$= \bar{v}_1 \times \cos v_{-1} + \bar{v}_2 \times v_0$	$= \cos 2 - 3$
v_0	$= x_2$	$= 3$	$\bar{v}_0$	$= \bar{v}_2 \times v_{-1}$	$= -1 \times 2$
v_1	$= \sin v_{-1}$	$= \sin 2$	$\bar{v}_1$	$= \bar{v}_3 \times 1$	$= 1 \times 1$
v_2	$= v_{-1} \times v_0$	$= 2 \times 3$	$\bar{v}_2$	$= \bar{v}_3 \times (-1)$	$= 1 \times (-1)$
v_3	$= v_1 - v_2$	$= \sin 2 - 6$	$\bar{v}_3$	$= \bar{y}_1$	$= 1$
y_1	$= v_3$	$= \sin 2 - 6$	$\bar{y}_1$		$= 1$

Both forward and reverse approaches indeed yield the same result:

$$\dot{y}_1 = \frac{\partial y_1}{\partial x_1} = \cos 2 - 3 \quad \text{(forward AD)}, \qquad \bar{v}_{-1} = \frac{\partial y_1}{\partial x_1} = \cos 2 - 3 \quad \text{(reverse AD)}.$$

This generalizes to $u : \mathbb{R}^d \to \mathbb{R}^{d'}$. Each pass is initialized by setting a single output derivative $\bar{y}_j = 1$, i.e.,

$$\bar{\boldsymbol{y}} = \boldsymbol{e}_j \in \mathbb{R}^{d'}.$$

A backward sweep then computes the corresponding input sensitivities

$$\bar{x}_i = \frac{\partial y_j}{\partial x_i}(\boldsymbol{a}), \quad i = 1, \ldots, d,$$

giving the j-th row of the Jacobian. The full Jacobian requires d' reverse passes, but products $J^\top \boldsymbol{r}$ can be computed in a single pass by setting $\bar{\boldsymbol{y}} = \boldsymbol{r}$.

Takeaways

Forward and reverse AD modes propagate derivatives in complementary directions:

- **Forward AD:** propagate $\dot{\boldsymbol{x}}$ from inputs to outputs; efficient when $d \ll d'$ or for computing $J\boldsymbol{r}$.
- **Reverse AD:** propagate $\bar{\boldsymbol{y}}$ from outputs to inputs; efficient when $d' \ll d$ or for computing $J^\top \boldsymbol{r}$.

Operation	Forward AD	Reverse AD
Compute full Jacobian J	$\mathcal{O}(d)$ passes	$\mathcal{O}(d')$ passes
Compute $J\boldsymbol{r}$	$\mathcal{O}(1)$ pass	$\mathcal{O}(d')$ passes
Compute $J^\top \boldsymbol{r}$	$\mathcal{O}(d)$ passes	$\mathcal{O}(1)$ pass

5 Exercises

Exercise 5.1 (Monte Carlo)**.** Let $f \in L^2([0,1]^d)$ and $\{\boldsymbol{x}_i\}_{i=1}^n$ be i.i.d. uniform in $[0,1]^d$. Define:

$$I = \int_{[0,1]^d} f(\boldsymbol{x})d\boldsymbol{x}, \qquad I_n = \frac{1}{n}\sum_{i=1}^n f(\boldsymbol{x}_i).$$

(a) Show that I_n is an unbiased estimator of I, i.e., $\mathbb{E}[I_n] = I$.

(b) Compute the variance $\mathrm{Var}(I_n)$ and deduce that the root-mean-square error satisfies

$$\mathbb{E}[|I - I_n|^2]^{1/2} = \mathcal{O}(n^{-1/2}) \qquad \text{independently of the dimension } d.$$

Exercise 5.2 (Linearized dynamics)**.** Consider the continuous loss

$$\mathcal{J}(\boldsymbol{\theta}) = \frac{1}{2}\|\mathcal{L}u_\theta - f\|^2_{L^2(D)} + \frac{\gamma}{2}\|u_\theta - g\|^2_{L^2(\partial D)},$$

and the gradient-descent iteration $\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^k - h\nabla\mathcal{J}(\boldsymbol{\theta}^k)$.

(a) Show that

$$\partial_{\theta_j}\mathcal{J}(\theta) = (\mathcal{L}u_\theta - f, \mathcal{L}\partial_{\theta_j}u_\theta)_{L^2(D)} + \gamma(u_\theta - g, \partial_{\theta_j}u_\theta)_{L^2(\partial D)}.$$

(b) Assuming that u_θ is approximately linearly in $\boldsymbol{\theta}$ near $\boldsymbol{\theta}^0$, and setting $\phi_i := \partial_{\theta_i}u_{\theta^0}$, show that

$$\begin{aligned}\partial_{\theta_j}\mathcal{J}(\theta) \approx{}& (\mathcal{L}u_{\theta^0} - f, \mathcal{L}\phi_j)_{L^2(D)} + \sum_{i=1}^p(\theta_i - \theta_i^0)(\mathcal{L}\phi_i, \mathcal{L}\phi_j)_{L^2(D)} \\ &+ \gamma(u_{\theta^0} - g, \phi_j)_{L^2(\partial D)} + \gamma\sum_{i=1}^p(\theta_i - \theta_i^0)(\phi_i, \phi_j)_{L^2(\partial D)}.\end{aligned}$$

(c) Conclude that the gradient descent iteration simplifies to $\boldsymbol{\theta}^{k+1} = (I - hA)\boldsymbol{\theta}^k + h(A\boldsymbol{\theta}^0 + \boldsymbol{b})$.

Exercise 5.3 (NTK)**.** Consider the discrete loss

$$J(\boldsymbol{\theta}) = \frac{1}{2}\sum_{i=1}^{n_r}\big|\mathcal{L}u_\theta(\boldsymbol{x}_i) - f(\boldsymbol{x}_i)\big|^2 + \frac{1}{2}\sum_{i=1}^{n_b}\big|u_\theta(\boldsymbol{z}_i) - g(\boldsymbol{z}_i)\big|^2.$$

and continuous-time limit of gradient descent $\dot{\boldsymbol{\theta}}(t) = -\nabla_\theta J(\boldsymbol{\theta}(t))$.

(a) Show that

$$\nabla J(\boldsymbol{\theta}) = \sum_{i=1}^{n_r}[\mathcal{L}u_\theta(\boldsymbol{x}_i) - f(\boldsymbol{x}_i)]\,\nabla_\theta\mathcal{L}u_\theta(\boldsymbol{x}_i) + \sum_{i=1}^{n_b}[u_\theta(\boldsymbol{z}_i) - g(\boldsymbol{z}_i)]\,\nabla_\theta u_\theta(\boldsymbol{z}_i).$$

(b) Deduce that, along the gradient flow,

$$\begin{aligned}\frac{d}{dt}\mathcal{L}u_{\theta(t)}(\boldsymbol{x}_i) = &-\sum_{j=1}^{n_r}[\mathcal{L}u_{\theta(t)}(\boldsymbol{x}_j) - f(\boldsymbol{x}_j)]\nabla_\theta\mathcal{L}u_{\theta(t)}(\boldsymbol{x}_i)^\top\nabla_\theta\mathcal{L}u_{\theta(t)}(\boldsymbol{x}_j) \\ &-\sum_{j=1}^{n_b}[u_{\theta(t)}(\boldsymbol{z}_j) - g(\boldsymbol{z}_j)]\nabla_\theta\mathcal{L}u_{\theta(t)}(\boldsymbol{x}_i)^\top\nabla_\theta u_{\theta(t)}(\boldsymbol{z}_j),\end{aligned}$$

and

$$\begin{aligned}\frac{d}{dt}u_{\theta(t)}(\boldsymbol{z}_i) = &-\sum_{j=1}^{n_r}[\mathcal{L}u_{\theta(t)}(\boldsymbol{x}_j) - f(\boldsymbol{x}_j)]\nabla_\theta u_{\theta(t)}(\boldsymbol{z}_i)^\top\nabla_\theta\mathcal{L}u_{\theta(t)}(\boldsymbol{x}_j) \\ &-\sum_{j=1}^{n_b}[u_{\theta(t)}(\boldsymbol{z}_j) - g(\boldsymbol{z}_j)]\nabla_\theta u_{\theta(t)}(\boldsymbol{z}_i)^\top\nabla_\theta u_{\theta(t)}(\boldsymbol{z}_j).\end{aligned}$$

6 Suggested reading

Check out the PINNs paper [12], building on [6]. Reviews on theory/practice are in [4, 9, 10, 13] and [2, 8, 14]. Improvements to PINNs include preconditioning [3], NTK perspective to mitigate spectral bias [11, 15], domain decomposition [5], and alternative architectures [7]. For automatic differentiation, see [1].

Table 1: *TL;DR table of 15 favorite references.*

	Key papers
PINNs original papers	[6, 12]
PINNs theory	[4, 9, 10, 13]
PINNs practice	[2, 8, 14]
Improving PINNs	[3, 5, 7, 11, 15]

References

[1] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, *Automatic differentiation in machine learning: a survey*, J. Mach. Learn. Res., 18 (2018), pp. 1–43.

[2] S. Cuomo, V. S. Di Cola, F. Giampaolo, G. Rozza, M. Raissi, and F. Piccialli, *Scientific machine learning through PINNs: Where we are and what's next*, J. Sci. Comput., 92 (2022).

[3] T. De Ryck, F. Bonnet, S. Mishra, and E. de Bézenac, *An operator preconditioning perspective on training in physics-informed machine learning*, in ICLR Proceedings, 2024.

[4] T. De Ryck and S. Mishra, *Numerical analysis of physics-informed neural networks and related models in physics-informed machine learning*, arXiv preprint, 2402.10926 (2024).

[5] A. Klawonn, M. Lanser, and J. Weber, *Machine learning and domain decomposition methods - a survey*, Comput. Sci. Eng., 1 (2024), pp. 1–31.

[6] I. E. Lagaris, A. Likas, and D. I. Fotiadis, *Artificial neural networks for solving ordinary and partial differential equations*, IEEE Trans. Neural Netw., 9 (1998), pp. 987–1000.

[7] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljacic, T. Y. Hou, and M. Tegmark, *KAN: Kolmogorov–Arnold networks*, arXiv preprint, 2404.19756 (2024).

[8] L. Lu, X. Meng, Z. Mao, and G. E. Karniadakis, *DeepXDE: A deep learning library for solving differential equations*, SIAM Rev., 63 (2021), pp. 208–228.

[9] S. Mishra and R. Molinaro, *Estimates on the generalization error of PINNs for approximating a class of inverse problems for PDEs*, IMA J. Numer. Anal., 42 (2022), pp. 981–1022.

[10] ——, *Estimates on the generalization error of physics-informed neural networks for approximating PDEs*, IMA J. Numer. Anal., 43 (2023), pp. 1–43.

[11] N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. Hamprecht, Y. Bengio, and A. Courville, *On the spectral bias of neural networks*, in ICML Proceedings, 2019.

[12] M. Raissi, P. Perdikaris, and G. E. Karniadakis, *PINNs: A deep learning framework for solving forward and inverse problems involving nonlinear pdes*, J. Comput. Phys., 378 (2019), pp. 686–707.

[13] J. W. Siegel, *Optimal approximation rates for deep ReLU neural networks on Sobolev and Besov spaces*, J. Mach. Learn. Res., 24 (2023), pp. 1–52.

[14] S. Wang, S. Samkaran, H. Wang, and P. Perdikaris, *An expert's guide to training physics-informed neural networks*, arXiv preprint, 2308.08468 (2023).

[15] S. Wang, X. Yu, and P. Perdikaris, *When and why PINNs fail to train: A neural tangent kernel perspective*, J. Comput. Phys., 449 (2022), p. 110768.